

MIRROR: Local Evidence Bundles for Reproducible Research Software Review

Issues in Research Software preprint candidate, v0.1

Anton Sokolov

Researcher, Tyche Institute, Tallinn, Estonia

ORCID: 0000-0003-2452-7096

30 May 2026

Status. Public-release preprint preview. Not peer reviewed, with no external venue submission, DOI deposit, signature, or timestamp.

Abstract

MIRROR is a small research-software method for keeping drafts, scripts, source copies, and claim records reviewable while an iterative research project is still moving. A MIRROR bundle places an artifact, structured claims, provenance notes, source descriptors or retained source copies, assumptions, and SHA-256 digests into a local directory that can be verified offline. This article reports the design as an experience paper for research software practice. Its central claim is deliberately narrow: byte-level replay and schema conformance can reduce the cost of later review, but they do not establish truth, completeness, authorship, legal compliance, public release authority, or regulated trust-service status. The case study is a private local workspace in which autonomous workers and human review produced draft papers, evidence packages, and daily roll-ups. The current implementation passed 166 local tests and a set of positive and negative verifier fixtures, including expected failures for hash mismatches and unsafe assurance language. The main contribution is not another assurance badge, but a boundary discipline: a reviewer should be able to ask what bytes were retained, which claim points to which evidence, what failed, and what still requires human judgment.

Keywords: research software; reproducibility; provenance; evidence bundles; artifact review; software sustainability

1 Introduction

Research software often fails quietly before it fails technically. A script still runs, but the draft that cites it has changed. A source was once available, but the retained copy is missing. A table is regenerated, but the claim it supports remains stronger than the evidence. In small projects this is annoying; in AI-governance and evidence-infrastructure work it becomes a methodological risk, because language such as “verified”, “attested”, or “compliant” can drift faster than the underlying evidence.

MIRROR started from that practical problem. It is not a new cryptographic primitive and not a replacement for repositories, archives, Research Object Crate (RO-Crate), PROV, software citation, or artifact evaluation practices (1-7). It is a deliberately modest layer that can be used before a project is ready for public release: gather the artifact, the retained evidence, the claims, and the assumptions into one replayable directory; then let an offline verifier answer only the question it is qualified to answer. Did the retained bytes and declared structure survive?

The venue fit for this preprint is the *Issues in Research Software* section of the Journal of Open Research Software (JORS). That section accepts experience reports and longer-form articles about creating, maintaining, and evaluating open source research software, asks for an abstract, encourages display items, uses Vancouver-style references, and suggests roughly 3000-4000 words (8). Recent JORS experience and policy papers use compact framing, explicit lessons, and back-matter statements rather than a heavy conference template (9-12). This draft follows that pattern while preserving one important caveat: the software and minimal public-safe evidence package must be easier to inspect than the prose that describes them.¹

2 Method

MIRROR models a reviewable artifact as a small tuple:

$$B = (a, S, C, N, M),$$

where a is the artifact being checked, S is the set of retained sources or source descriptors, C is a structured claim register, N is a human-readable note file, and M is a manifest containing schema version, hashes, assumptions, and non-signing metadata. The reference verifier implements the narrow predicate:

$$V(B) = \text{ok} \iff \text{schema}(M, C) \wedge H(a) = M_a \wedge \forall s \in S : H(s) = M_s \wedge \text{policy}(C, N),$$

where H is SHA-256 and policy is a local guardrail over configured wording and pointer rules.

Lemma 1: Byte replay is not claim truth. If $V(B) = \text{ok}$, then the retained local files match the recorded hashes and schemas. It does not follow that the research claim is correct, that the retained source is sufficient, or that a public release is safe. The lemma is almost embarrassingly simple, which is why it is useful. It stops the verifier from becoming a badge with better typography.²

The implementation is intentionally ordinary Python. The repository contains a bundler, an offline verifier, schema files, daily roll-up tooling, a signed-transition profile generator that does not sign, and positive and negative fixtures. The test suite covers schema validity, hash mismatch, missing files, unsafe assurance wording, roll-up payload semantics, local anchor-chain validation, reviewer-report packets, and signed-transition profile payloads. A test run on 30 May 2026 completed 166 tests successfully.

Layer	What MIRROR records	What it refuses to infer
Bundle	Artifact bytes, retained sources, claims, notes, assumptions, and digests.	Truth, novelty, fairness of interpretation, or completeness of evidence.
Verifier	Schema conformance, digest equality, required entries, and configured policy failures.	Authorship, peer review, legal compliance, public release readiness, or regulated assurance.
Roll-up	Local artifact-index rows, digest observations, diagnostic categories, and a local Merkle root.	Lane health scores, publication decisions, audits of sibling projects, or external time.
Anchor chain	A hash-linked JSONL sequence over retained local roll-up records.	Notarization, priority proof, certified timestamping, or identity proof.

Table 1: Interpretation boundaries for the current MIRROR workflow. The table is intentionally negative as well as positive: the refusal column is part of the method.

3 Worked Case

The case study is the MIRROR lane inside a private Tyche Institute research workspace. The lane produced a schema, reference verifier, bundler, daily roll-up tool, examples, release-candidate notes, and a draft methodology paper. It also read sibling artifact indexes in read-only mode and wrote advisory tickets locally when rows were missing, stale, or internally conflicting. Those tickets were not sent to other projects and did not modify sibling lanes.

The smallest successful example is a retained artifact plus one source file and one low-risk claim. Running the verifier on that bundle returns a machine-readable status:

```
{"errors": [], "status": "ok"}
```

The smallest failure example changes a source byte after bundling. The verifier reports a hash mismatch and returns **fail**. That failure is not an accusation; it is a pointer. A reviewer can ask whether the edit was expected, whether the bundle should be regenerated, or whether the claim should be narrowed.

This failed case is intentionally plain. It models the common maintenance accident in which a retained source, scraped excerpt, or generated table is updated after the prose has already absorbed its earlier form. The interesting result is not that hashing detects the byte change. The interesting result is that the verifier keeps the discussion at the right level: “which evidence moved, which claim depends on it, and what decision now belongs to the author or reviewer?” A negative fixture therefore becomes a small rehearsal for peer review rather than a dramatic security incident.

Daily roll-ups add a second layer. They scan configured sibling indexes, normalize rows, compute local roots, and append anchor records whose `external_timestamp` value remains null. That null is not a bug. It is a boundary

marker: the local chain supports replay of retained local records, not an independent time claim. This makes MIRROR adjacent to content provenance systems such as C2PA, software supply-chain attestations such as SLSA and in-toto, and signing ecosystems such as Sigstore, but it does not claim their trust model or public-log semantics (13-16).

The same distinction matters vertically. MIRROR uses SHA-256 as a conservative byte-integrity primitive under the Secure Hash Standard, not as an oracle about scholarly truth (17). Public-key infrastructure (PKI), the X.509 certificate profile, the European eIDAS trust-services regulation, and European Telecommunications Standards Institute (ETSI) AdES validation procedures live higher in the governance stack (18-20). MIRROR prepares material that such systems could later sign, seal, time-stamp, or reject; it does not borrow their authority in advance.⁴

Adjacent practice	Shared concern	MIRROR distinction
PROV and RO-Crate	Describe provenance relationships and package research objects.	Uses a smaller file-system-native packet for claim review before public packaging.
FAIR and FAIR4RS	Make research outputs findable, accessible, interoperable, and reusable.	Focuses on local challengeability before the output is ready to be released.
Software citation	Make software a citable research product.	Records claim-evidence links around the software, not only software identity.
Artifact badging	Evaluate availability, functionality, reuse, or reproduced results.	Produces no badge and deliberately avoids quality grading.
Attestation systems	Bind artifacts to build steps, identities, logs, or provenance predicates.	Keeps v0.1 unsigned and offline; future signing is a separate operator-controlled layer.

Table 2: MIRROR is deliberately near existing provenance and software-sustainability work, but its unit of review is a claim-bearing local research artifact.

4 Lessons for Research Software Practice

The first lesson is that negative semantics need first-class space. Many tools describe what a successful check means; fewer force the author to write what it does not mean. In MIRROR, every green result travels with a non-certification boundary. This is not decorative caution. It prevents a local maintenance check from being confused with a publication decision, conformity assessment, or legal opinion.

The second lesson is that claim registers are more useful than large undifferentiated supplements. A reviewer does not need every file at once. The better question is: which claim is being inspected, what evidence pointer supports it, what is the risk level, and what safe wording would survive if the evidence is weaker than hoped? This is close in spirit to artifact evaluation, but it moves the review unit from “the whole artifact works” to “this statement has a replay path” (7).

The third lesson is that failures should stay boring. A hash mismatch may be important, but it may also mean that a worker edited a file and forgot to refresh the bundle. MIRROR therefore emits deterministic errors, not moral adjectives. The human reviewer still has to decide whether the mismatch is harmless maintenance debt, a publication blocker, or a sign that the claim no longer belongs in public text.

The fourth lesson is that local evidence packages help precisely because they are weaker than public attestations. A private workspace cannot always publish every source, copy every page, or expose every intermediate file. MIRROR can still improve the situation by recording retained bytes, descriptors, assumptions, and blocked-publication notes. That is less glamorous than a public cryptographic log, but it is available at the moment when many mistakes are still cheap to fix.

The fifth lesson is that future signing should be additive. The repository already includes a signed-transition profile generator that hashes an unsigned bundle and a reviewer-report packet, then writes a profile for later operator-controlled signing. It performs no signing itself. This keeps EATF-style or other attestation layers separate from the local replay substrate.³

5 Limitations and Threat Model

MIRROR assumes that the local filesystem, retained files, and verifier code are available for inspection. It does not defend against a malicious operating system, hidden key compromise, forged source authority, social pressure, selective omission of claims, or a reviewer who does not read the evidence. It also does not solve licensing. A file may be stable and still not be redistributable; a source descriptor may be honest and still be too weak for a public claim.

The current case is a self-experiment. That is both a strength and a limitation. It gives unusually direct access to iteration history and design choices, but it does not show how independent teams would use the method, how reviewers would score its usefulness, or how it behaves across public repositories with different licensing and contribution cultures. The public-release surface now needs to be treated as part of the method: repository metadata, license text, citation metadata, reviewer instructions, release notes, and a minimal evidence package are not administrative extras; they are the first reproducibility claim the paper makes.

The security claims are also narrow. SHA-256 digests support byte identity. JSON Schema supports structure. Local policy checks catch configured patterns of unsafe wording. None of these proves that a claim is right. The best interpretation is maintenance-oriented: MIRROR helps make the next human review sharper, faster, and more reproducible.

6 Conclusion

MIRROR is a small answer to a common research-software problem: evidence moves, claims harden, and later reviewers inherit a pile of plausible files. The method binds artifacts, sources, claims, notes, and digests into an offline packet, then insists on a modest interpretation of the result. That modesty is the point. A useful verifier should help a reviewer challenge a claim without pretending to decide the claim.

For JORS, the strongest version of this article is an experience report about research software maintenance under uncertainty. Its practical contribution is a workflow and vocabulary for local evidence bundles before public release; its theoretical contribution is a clean separation between byte replay and scholarly judgment. The next step is not stronger prose. It is the remaining publication hygiene that the paper itself demands: a DOI archive if the venue or reviewers expect one, and a final check that the public release still says less, not more, than the verifier can prove.

Data Accessibility

The software, tests, fixtures, and minimal public-safe evidence package for this preprint are available in the public release repository at <https://github.com/sapsan14/mirror-evidence-bundles> under the Apache License 2.0. The minimal evidence package is in `release/mirror-methodology-rc1/`; the current release target is `v0.1.0-jors-preprint`. No Zenodo, OSF, or journal DOI is claimed in this PDF. If a DOI archive is created before journal submission, this statement should be updated to cite the DOI rather than the GitHub URL alone.

Ethics and Consent

The case study uses the author's own local research workspace and software artifacts. It does not report human-subjects data, private third-party data, customer data, or employer-confidential material. The public version must preserve that boundary by excluding private paths, unpublished third-party source material, and any text that implies operational access to external systems.

Funding Information

No external funding is claimed for this preprint.

Competing Interests

The author declares no competing interests for this preprint. The author has professional experience in public-key infrastructure and digital identity systems; no employer, client, or non-public operational material is named or used in this article.

Author Contributions

Anton Sokolov designed the MIRROR workflow, implemented or supervised the local reference tooling, ran the local tests and verifier checks, selected the venue route, and prepared this manuscript.

AI Assistance Disclosure

This preprint was prepared with substantive generative-AI assistance. The venue-selection, restructuring, reference triage, and final preprint build were assisted by OpenAI Codex in dialogue with the author, using local project files and public venue/source checks. The author remains responsible for all claims, references, omissions, and the decision whether any public submission is appropriate.

Notes

1. JORS software metapapers require freely available software in a public repository. The present draft is routed instead to the JORS *Issues in Research Software* article type, but public code, license, and archive decisions still matter because the article introduces a software method.
2. A hash is wonderfully stubborn about bytes and wonderfully silent about meaning. That is a feature only if the paper says so plainly.
3. EATF here means an evidence or agent attestation framework layer that could later sign or publish a handoff payload. The current MIRROR preprint does not claim that such signing has occurred.
4. PKI here means public-key infrastructure: the people, policies, certificates, keys, revocation records, software, and institutional habits that make a signature mean more than “some private key produced these bytes”. The cryptography is hard; the governance is where it quietly becomes interesting.

References

1. Wilkinson MD, Dumontier M, Aalbersberg IJ, Appleton G, Axton M, Baak A, et al. The FAIR Guiding Principles for scientific data management and stewardship. *Scientific Data*. 2016;3:160018. DOI: <https://doi.org/10.1038/sdata.2016.18>.
2. Barker M, Chue Hong NP, Katz DS, Lamprecht AL, Martinez C, Psomopoulos F, et al. Introducing the FAIR Principles for research software. *Scientific Data*. 2022;9:622. DOI: <https://doi.org/10.1038/s41597-022-01710-x>.
3. Smith AM, Katz DS, Niemeyer KE, FORCE11 Software Citation Working Group. Software citation principles. *PeerJ Computer Science*. 2016;2:e86. DOI: <https://doi.org/10.7717/peerj-cs.86>.
4. Groth P, Moreau L, editors. PROV-Overview: An Overview of the PROV Family of Documents. W3C Working Group Note; 2013 Apr 30. Available from: <https://www.w3.org/TR/prov-overview/>.
5. Sefton P, Carragain EO, Soiland-Reyes S, Corcho O, Garijo D, Palma R, et al. RO-Crate Metadata Specification 1.1. researchobject.org community; 2023 Apr 26. DOI: <https://doi.org/10.5281/zenodo.7867028>.
6. Mesirov JP. Accessible reproducible research. *Science*. 2010;327(5964):415-416. DOI: <https://doi.org/10.1126/science.1179653>.
7. Association for Computing Machinery. Artifact Review and Badging Version 1.1. ACM Publications Policy; 2020 Aug 24. Available from: <https://www.acm.org/publications/policies/artifact-review-and-badging-current>.

8. Journal of Open Research Software. Submission Guidelines. Ubiquity Press; cited 2026 May 30. Available from: <https://openresearchsoftware.metajnl.com/about/submissions>.
9. Fehr J, Himpe C, Rave S, Saak J. Sustainable Research Software Hand-Over. Journal of Open Research Software. 2021;9(1):5. DOI: <https://doi.org/10.5334/jors.307>.
10. Lamprecht AL, Martinez-Ortiz C, Barker M, Bartholomew SL, Barton J, Chue Hong N, et al. What Do We (Not) Know About Research Software Engineering? Journal of Open Research Software. 2022;10(1):10. DOI: <https://doi.org/10.5334/jors.384>.
11. Katz DS, Druskat S, Haines R, Jay C, Struck A. The State of Sustainable Research Software: Learning from the Workshop on Sustainable Software for Science: Practice and Experiences (WSSSPE5.1). Journal of Open Research Software. 2019;7(1):11. DOI: <https://doi.org/10.5334/jors.242>.
12. Barker M, Cohen J, Hernandez Serrano P, Katz DS, Martin K, Rudmann D, Shanahan H. Institutional Policy Pathways for Supporting Research Software: Global Trends and Local Practices. Journal of Open Research Software. 2025;13:39. DOI: <https://doi.org/10.5334/jors.625>.
13. Supply-chain Levels for Software Artifacts. SLSA specification version 1.2. Linux Foundation; cited 2026 May 30. Available from: <https://slsa.dev/spec/v1.2/>.
14. Torres-Arias S, Afzali H, Kuppusamy TK, Curtmola R, Cappos J. in-toto: Providing farm-to-table guarantees for bits and bytes. In: Proceedings of the 28th USENIX Security Symposium; 2019; Santa Clara, CA. Available from: <https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias>.
15. Coalition for Content Provenance and Authenticity. C2PA Specifications 2.1. Joint Development Foundation; cited 2026 May 30. Available from: <https://spec.c2pa.org/specifications/specifications/2.1/index.html>.
16. Sigstore. Overview. OpenSSF / Linux Foundation; cited 2026 May 30. Available from: <https://docs.sigstore.dev/>.
17. National Institute of Standards and Technology. Secure Hash Standard (SHS), FIPS PUB 180-4. Gaithersburg, MD: NIST; 2015 Aug. DOI: <https://doi.org/10.6028/NIST.FIPS.180-4>.
18. Cooper D, Santesson S, Farrell S, Boeyen S, Housley R, Polk W. Internet X.509 Public Key Infrastructure Certificate and Certificate Revocation List (CRL) Profile. RFC 5280. IETF; 2008 May. DOI: <https://doi.org/10.17487/RFC5280>.
19. European Parliament and Council. Regulation (EU) No 910/2014 on electronic identification and trust services for electronic transactions in the internal market and repealing Directive 1999/93/EC. Official Journal of the European Union. 2014;L257:73-114. Available from: <https://eur-lex.europa.eu/eli/reg/2014/910/oj/eng>.
20. ETSI. ETSI EN 319 102-1 V1.4.1: Electronic Signatures and Infrastructures (ESI); Procedures for Creation and Validation of AdES Digital Signatures; Part 1: Creation and Validation. Sophia Antipolis: ETSI; 2024 Jun. Available from: https://www.etsi.org/deliver/etsi_en/319100_319199/31910201/01.04.01_60/en_31910201v010401p.pdf.